DSA STUDY BLUEPRINT SERIES

47. Count Inversions Problem

Counting Inversions in Array using Merge Sort

Section 06 • C++ Core Data Structures & Algorithms

Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Inversion:** Pair of indices (*i*, *j*) where `i < j` and `arr[i] > arr[j]`.
- Inversions are counted while merging elements during Merge Sort.

Memory & Execution Blueprint

Left sorted: [2, 4] | Right sorted: [1, 3] ⇒ 2 > 1 (Count = 2 inversions)

C++ Implementation Reference

Standard code layout and syntax

```
long long mergeAndCount(int arr[], int l, int m, int r) {
    // Merge logic; if L[i] > R[j], all remaining elements in L
    // form inversions with R[j]. Add count += (mid - i + 1).
    return count;
}
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Time Complexity	O(N log N)
Space Complexity	O(N)

Key Practices & Gotchas

- **Important:** Inversion counts indicate how close an array is to being fully sorted.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's dry run the partition splitting sequence for Merge Sort on list `[4, 2, 1, 3]`:

Depth Level	Divided segments	Sorting result	Merged result
1	[4, 2] & [1, 3]	Recursive sorting	[2, 4] & [1, 3] merged to [1, 2, 3, 4]

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Pivot heuristics:** Quick Sort speeds depend on pivot balance; choosing middle or random elements prevents O(N^2) degradation.
- **Inversions:** Count array inversions during merge sorts to identify out-of-order bounds sizes.
- **Related LeetCode Problems:** #912 Sort an Array, #315 Count of Smaller Numbers After Self.